

Caso: Avaliação de classificador binário na sklearn

Aula 07 – Inteligência Computacional

Eduardo Corrêa Gonçalves

09/09/2026

Sumário

Avaliação Holdout

- Passos para avaliar um classificador binário com holdout na sklearn

- Exemplo passo a passo

Comparação do desempenho de modelos:

- CART com hiperparâmetros default

- CART com configuração de hiperparâmetros

- Random Forest com parâmetros default

- Regressão Logística com parâmetros default

Avaliação por Validação Cruzada

- Passos para avaliar um classificador binário com validação cruzada na sklearn

Link para código com o exemplo da aula

- **Caso – Avaliação de classificador binário de sátiras na sklearn: holdout e validação cruzada (exemplo dessa aula)**
 - <https://colab.research.google.com/drive/15O4RXW0PxoXI1HuN0CHRBnhyiWEGkTf1?usp=sharing>

Introdução (1/2)

Os 6 passos para avaliar um classificador por holdout na sklearn:

1. Importação da base de dados rotulada.
2. Separação das partes X (*features*) e Y (classe).
3. Pré-processamento da parte X .
4. Divisão da base rotulada em treino e teste
5. Treinamento do modelo com a partição de treino.
6. Teste do modelo na partição de teste.

Não é um esquema fixo! Dependendo do problema, você pode alterar esse fluxo.

- **Exemplo1:** Algumas vezes a base rotulada já é fornecida com as partições de treino e teste separadas. Neste caso, você não precisará do passo 4.
- **Exemplo2:** muitas vezes é necessário realizar o pré-processamento na base de treino (`fit_transform`) e depois aplicar (`transform`) na base de teste.
 - Nessa situação, é preciso dividir em treino e teste antes de pré-processar.

Introdução (2/2)

Vamos demonstrar cada etapa utilizando como exemplo uma árvore de decisão para **classificação de sátiras**:

Classificador que “lê” um vetor de features extraídos do texto de uma notícia e diz se ela é “normal” (classe 0) ou “sátira” (classe 1)

Nesse caso, o pré-processamento do texto já foi realizado...

	satira	qtd_excl	qtd_pont	qtd_adv_intens	qtd_interj	qtd_adj_pos	qtd_adj_neg	qtd_pal_pos	qtd_pal_neg	qtd_pal_neu	...	raz_PRON	raz_PROPN	raz_PUNCT	raz_SC
0	0	0	6	1	0	1	0	3	1	6	...	0.012821	0.192308	0.076923	0.038
1	0	0	7	0	0	0	0	1	3	5	...	0.020833	0.291667	0.145833	0.000
2	0	0	6	0	0	0	0	3	0	3	...	0.016393	0.098361	0.098361	0.000
3	0	0	18	1	0	1	3	5	6	1	...	0.035088	0.131579	0.157895	0.008
4	0	0	10	1	0	1	1	2	2	10	...	0.000000	0.000000	0.161290	0.032
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
18489	1	0	31	2	0	5	3	9	9	18	...	0.012987	0.082251	0.134199	0.043
18490	1	0	29	1	0	2	2	4	3	11	...	0.029586	0.124260	0.171598	0.035
18491	1	0	25	1	0	2	2	9	5	11	...	0.017857	0.190476	0.148810	0.017
18492	1	0	27	0	0	3	0	7	4	19	...	0.028846	0.153846	0.129808	0.043
18493	1	0	30	2	0	1	1	7	5	14	...	0.044776	0.049751	0.149254	0.009

Avaliação holdout na sklearn: passo a passo (1/14)

PASSO 1: importação da base de dados (BD) rotulada

- Usaremos a base CSV produzida no notebook anterior
 - Possui 41 colunas:
 - na 1ª a classe
 - nas seguintes os 40 atributos preditivos, todos numéricos.
 - São 18.493 observações, sendo 11272 negativas (textos normais) e 7222 positivas (sátiras)
- O arquivo CSV será importado para um **DataFrame** pandas (código abaixo).

```
import pandas as pd

# -----
# Importa os dados de treinamento para um DataFrame
# -----
arq =
'https://raw.githubusercontent.com/edubd/ic/refs/heads/main/bd_sat
iras2_estruturado.csv'
df = pd.read_csv(arq)

df # para visualizar o conteúdo
```

Avaliação holdout na sklearn: passo a passo (2/14)

Resultado ao final do PASSO 1

BD rotulada terá sido importado p/ DataFrame

	satira	qtd_excl	qtd_pont	qtd_adv_intens	qtd_interj	qtd_adj_pos	qtd_adj_neg	qtd_pal_pos	qtd_pal_neg	qtd_pal_neu	...	raz_PRON	raz_PROPN	raz_PUNCT	raz_SC
0	0	0	6	1	0	1	0	3	1	6	...	0.012821	0.192308	0.076923	0.038
1	0	0	7	0	0	0	0	1	3	5	...	0.020833	0.291667	0.145833	0.000
2	0	0	6	0	0	0	0	3	0	3	...	0.016393	0.098361	0.098361	0.000
3	0	0	18	1	0	1	3	5	6	1	...	0.035088	0.131579	0.157895	0.008
4	0	0	10	1	0	1	1	2	2	10	...	0.000000	0.000000	0.161290	0.032
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
18489	1	0	31	2	0	5	3	9	9	18	...	0.012987	0.082251	0.134199	0.043
18490	1	0	29	1	0	2	2	4	3	11	...	0.029586	0.124260	0.171598	0.035
18491	1	0	25	1	0	2	2	9	5	11	...	0.017857	0.190476	0.148810	0.017
18492	1	0	27	0	0	3	0	7	4	19	...	0.028846	0.153846	0.129808	0.043
18493	1	0	30	2	0	1	1	7	5	14	...	0.044776	0.049751	0.149254	0.009

Avaliação holdout na sklearn: passo a passo (3/14)

PASSO 2: separação das partes X (*features*) e Y (classe)

Como sabemos, para treinar o modelo a sklearn exige que seja indicado quem é o conjunto de *features* X e quem é a classe Y. Então, p/ facilitar, fazemos a separação

É resolvida com a operação de **fatiamiento de DataFrames** *que será detalhada em aula futura*.

```
X = df.iloc[:, 1:]      # Gera DataFrame com todas as linhas e
                        # as colunas 1 a 40
Y = df.iloc[:, 0]       # Gera Series todas as linhas
                        # e a coluna 0 (satira)

print('- X:'); print(X)
print()
print('- Y:'); print(Y)
```


Avaliação holdout na sklearn: passo a passo (4/14)

Resultado ao final do PASSO 2

X (matriz de features) e Y (vetor de classes) terão sido separados.

```
- Y:
0      0
1      0
2      0
3      0
4      0
...
18489   1
18490   1
18491   1
18492   1
18493   1
Name: satira, Length: 18494, dtype: int64
```

```

0      qtd_excl  qtd_pont  qtd_adv_intens  qtd_interj  qtd_adj_pos  \
0          0         6          1          0          1
1          0         7          0          0          0
2          0         6          0          0          0
3          0        18          1          0          1
4          0        10          1          0          1
...      ...      ...      ...      ...      ...
18489      0        31          2          0          5
18490      0        29          1          0          2
18491      0        25          1          0          2
18492      0        27          0          0          3
18493      0        30          2          0          1

0      qtd_adj_neg  qtd_pal_pos  qtd_pal_neg  qtd_pal_neu  qtd_conj_adv  ...  \
0          0         3          1          6          0  ...
1          0         1          3          5          0  ...
2          0         3          0          3          0  ...
3          3         5          6          1          0  ...
4          1         2          2         10          0  ...
...      ...      ...      ...      ...      ...  ...
18489      3         9          9         18          0  ...
18490      2         4          3         11          0  ...
18491      2         9          5         11          0  ...
18492      0         7          4         19          0  ...
18493      1         7          5         14          1  ...

0      raz_PRON  raz_PROPN  raz_PUNCT  raz_SCONJ  raz_SYM  raz_VERB  \
0      0.012821  0.192308  0.076923  0.038462  0.0  0.102564
1      0.020833  0.291667  0.145833  0.000000  0.0  0.083333
2      0.016393  0.098361  0.098361  0.000000  0.0  0.098361
3      0.035088  0.131579  0.157895  0.008772  0.0  0.061404
4      0.000000  0.000000  0.161290  0.032258  0.0  0.112903
...      ...      ...      ...      ...      ...  ...
18489  0.012987  0.082251  0.134199  0.043290  0.0  0.086580
18490  0.029586  0.124260  0.171598  0.035503  0.0  0.112426
18491  0.017857  0.190476  0.148810  0.017857  0.0  0.119048
18492  0.028846  0.153846  0.129808  0.043269  0.0  0.129808
18493  0.044776  0.049751  0.149254  0.009950  0.0  0.109453

0      qtd_pal_agradec  qtd_pal_desculp  qtd_perguntas  val_emotiv
0          0          0          0          0.200000
1          0          0          0          0.166667
2          0          0          0          0.136364
3          0          0          0          0.357143
4          0          0          0          0.523810
...      ...      ...      ...      ...
18489      0          0          0          0.441176
18490      0          0          0          0.230769
18491      0          0          0          0.266667
18492      0          0          0          0.212121
18493      0          0          1          0.317460

[18494 rows x 40 columns]
```

Avaliação holdout na sklearn: passo a passo (5/14)

PASSO 3: pré-processamento da parte X

Nesse exemplo não será necessário, pois todo pré-processamento (engenharia de características) foi feito no notebook anterior.

Avaliação holdout na sklearn: passo a passo (6/14)

PASSO 4: divide a base de dados em treino e teste

Utiliza-se o método `train_test_split` (classe `model_selection` do módulo `sklearn.model_selection`)

Ela **vai gerar 4 estruturas**, já estratificadas por padrão (respeitando a proporção das classes):

`X_treino` e `Y_treino` (que usaremos para treinar o classificador)

`X_teste` e `Y_teste` (que usaremos para testar o classificador)

Os parâmetros `train_size` e `test_size` são usados para definir o tamanho das partições de treino e teste, respectivamente (nesse caso 67% e 33%)

```
import sklearn.model_selection as model_selection

X_treino, X_teste, Y_treino, Y_teste =
model_selection.train_test_split(X, Y,
                                train_size=0.67,
                                test_size=0.33,
                                random_state=0)

print ("X treino: ", X_treino.shape)
print ("Y treino: ", Y_treino.shape)
print ("X teste: ", X_teste.shape)
print ("Y teste: ", Y_teste.shape)
```

Avaliação holdout na sklearn: passo a passo (7/14)

Resultado ao final do PASSO 4

X dividido em X_treino e X_teste; Y dividido em Y_treino e Y_teste

```
print ("X treino: ", X_treino.shape)
print ("Y treino: ", Y_treino.shape)
print ("X teste: ", X_teste.shape)
print ("Y teste: ", Y_teste.shape)
```

```
X_treino: (12390, 40)
Y_treino: (12390,)
X_teste: (6104, 40)
Y_teste: (6104,)
```

Avaliação holdout na sklearn: passo a passo (8/14)

Resultado ao final do PASSO 4

Verificando a **estratificação**

```
print('demonstrando a estratificação:')  
  
print('\nfreqs. labels de treino:');  
print(Y_treino.value_counts() * 100 / len(Y_treino))  
  
print('\nfreqs. labels de teste:');  
print(Y_teste.value_counts() * 100 / len(Y_teste))
```

```
demonstrando a estratificação:  
  
freqs. labels de treino:  
satira  
0    61.323648  
1    38.676352  
Name: count, dtype: float64  
  
freqs. labels de teste:  
satira  
0    60.190039  
1    39.809961  
Name: count, dtype: float64
```

Avaliação holdout na sklearn: passo a passo (9/14)

PASSO 5: treinamento do classificador com os dados de treino

- (1). **importar o pacote** com o algoritmo que você quer usar, nesse caso CART (pacote DecisionTreeClassifier da scikit)
- (2). **Instanciar o classificador**, indicando os parâmetros que você deseja.
Instanciar é como carregar o algoritmo em memória para posterior utilização. Utilizaremos os hiperparâmetros default.
- (3). Usar o método (função) **fit**, para fazer o **treinamento do classificador**, indicando X_treino e Y_treino.

```
# (1). importa a classe DecisionTreeClassifier do módulo
# sklearn.tree
from sklearn.tree import DecisionTreeClassifier

# (2). instancia (cria um objeto) DecisionTreeClassifier
#       sem mexer nos hiperparâmetros default
#       (apenas 'random state = 0' p/ gerar sempre a mesma AD)
modelo = DecisionTreeClassifier(random_state=0)

# (3). treina o modelo de AD CART com o método fit(),
#       indicando X e Y de treino
modelo.fit(X_treino, Y_treino)
```

Avaliação holdout na sklearn: passo a passo (10/14)

Resultado ao final do PASSO 5

Um modelo de classificação AD
CART treinado em memória,
treinado com os dados de
treino (X_treino, Y_treino)

Essa **AD** é bem grande !!!

altura = 20

294 regras p/ classe 0

291 regras p/ classe 1

total de **585 regras**

```

|--- tot_frases <= 4.50
| |--- tot_frases <= 3.50
| | |--- med_pal_par <= 13.50
| | | |--- class: 1
| | |--- med_pal_par > 13.50
| | |--- raz_VERB <= 0.13
| | | |--- med_comp_pal <= 4.72
| | | |--- med_pal_par <= 17.50
| | | |--- class: 1
| | | |--- med_pal_par > 17.50
| | | |--- med_pal_fra <= 21.83
| | | |--- raz_PUNCT <= 0.07
| | | |--- raz_CCONJ <= 0.03
| | | |--- raz_VERB <= 0.12
| | | |--- qtd_pal_neu <= 6.50
| | | |--- class: 1
| | | |--- qtd_pal_neu > 6.50
| | | |--- class: 0
| | | |--- raz_VERB > 0.12
| | | |--- class: 0
| | | |--- raz_CCONJ > 0.03
| | | |--- class: 0
...
| | | |--- raz_NOUN > 0.17
| | | |--- class: 0
| | | |--- raz_PUNCT > 0.19
| | | |--- class: 0

```

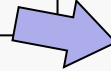
Avaliação holdout na sklearn: passo a passo (11/14)

PASSO 6: teste do modelo com os dados de teste

(6.1) primeiro temos que aplicar o classificador na partição X_teste.

Assim vamos obter as **predições do modelo** para **todos** os objetos de teste !!!

```
Y_predito = modelo.predict(X_teste)
```



0	0
1	1
2	0
3	1
4	0
...	...
6099	0
6100	1
6101	1
6102	1
6103	0

Avaliação holdout na sklearn: passo a passo (12/14)

PASSO 6: teste do modelo com os dados de teste

(6.2) agora vamos mandar a matriz de confusão ser gerada

A sklearn faz isso comparando o Y_predito com o Y de teste (classes reais)

0	0
1	1
2	0
3	1
4	0
...	...
6099	0
6100	1
6101	1
6102	1
6103	0

Y_predito

0
1
0
1
0
...
0
1
1
1
0

Y_teste

- *Curiosamente para os 10 objetos ao lado (5 primeiros e 5 últimos objetos de teste)...*
- *... Veja que todas as classificações foram corretas !!!*

Avaliação holdout na sklearn: passo a passo (13/14)

PASSO 6: teste do modelo com os dados de teste (cont...)

(6.2) de posse de Y_predito e Y_teste, podemos gerar a matriz de confusão

Usa-se a função `confusion_matrix` do módulo `metrics`.

A diagonal principal tem os maiores valores, o que significa que o classificador acertou a maioria das classificações

```
from sklearn import metrics  
# matriz de confusão  
mc = metrics.confusion_matrix(y_true = Y_teste,  
                               y_pred = Y_predito)
```

mc

Matriz de Confusão

[[3408	266]
[293	2137]]

A 1a linha/col é referente à classe 0 (notícia normal) e a 2a linha/col à classe 1 (notícia satírica)
Já que o Python usa ordena os rótulos de classe alfabeticamente

o formato da matriz neste exemplo é:

[[VN	FP]
[FN	VP]]

Avaliação holdout na sklearn: passo a passo (14/14)

PASSO 6: teste do modelo com os dados de teste (cont...)

(6.3) para obter os valores de acurácia, precisão, revocação e f1, basta usar as funções que calculam esse medida

Temos que passar como parâmetros:

- Y_teste (as classes reais),

- Y_predito (as classes que o modelo inferiu)

- Qual é a classe positiva do problema (no parâmetro pos_label)

- E especificar que estamos trabalhando com classificação binária (no parâmetro binary)

Veja o exemplo abaixo para o cálculo da precisão

```
# precisão = VP / (VP + FP)
precision = metrics.precision_score(Y_teste, Y_predito,
                                   pos_label = 1,
                                   average = 'binary')
```



Precisão:
0.889

Comparando com outros modelos (1/2)

Na página a seguir, uma tabela comparativa

Métodos:

CART com hiperparâmetros default

CART com `max_depth = 5` e `min_samples_leaf = 5`

Random Forest (RF) com hiperparâmetros default

Regressão Logística (RL) com hiperparâmetros default*

Medidas:

Acurácia, Precisão, Revocação, F1

* Observações quanto à regressão logística:

- A princípio, a regressão logística estaria sendo prejudicada pelo fato de os atributos preditivos não estarem numa mesma escala.
- Por padrão, executa 100 épocas.

Comparando com outros modelos (2/2)

Tabela comparativa – Avaliação de 4 modelos por holdout

Método	Acurácia	Precisão	Recall	F1
CART (default)	0,908	0,889	0,879	0,884
CART (configurado)	0,926	0,923	0,889	0,906
Random Forest (default)	0,941	0,942	0,907	0,925
Regressão Logística (default)	0,930	0,928	0,892	0,910

Avaliação por validação cruzada na sklearn (1/8)

Os 6 passos para avaliar um classificador por validação cruzada na sklearn:

1. Importação da base de dados rotulada.
2. Separação das partes X (*features*) e Y (classe).
3. Pré-processamento da parte X.
4. Estimativa por validação cruzada

Como sempre... não é um esquema fixo! Dependendo do problema, você pode alterar esse fluxo.

- **Exemplo:** muitas vezes é necessário realizar o pré-processamento nas partições de treino (`fit_transform`) e depois aplicar (`transform`) na partição de teste.
 - Nessa situação, isso precisa ser feito durante o processo de validação cruzada

Avaliação por validação cruzada na sklearn (2/8)

Vamos direto para o passo 4, pois os outros são iguais ao do holdout

1. Importação da base de dados rotulada.
2. Separação das partes X (*features*) e Y (classe).
3. Pré-processamento da parte X.
4. **Estimativa por validação cruzada**
 - Faremos o exemplo com o **RandomForest**

Avaliação por validação cruzada na sklearn (3/8)

PASSO 4: estimativa por validação cruzada

- Na scikit-learn: podemos usar a função `cross_validate` do módulo `model_selection`
- Ela fará o processo automaticamente, treinando e testando os k modelos e gerando um dicionário com os resultados gerais.

(1) primeiro importamos os pacotes necessários

```
from sklearn.model_selection import cross_validate
from sklearn.metrics import make_scorer
from sklearn.metrics import recall_score
from sklearn.metrics import precision_score
from sklearn.metrics import f1_score
from sklearn.metrics import accuracy_score
```


Avaliação por validação cruzada na sklearn (4/8)

PASSO 4: estimativa por validação cruzada

(2) instanciamos o classificador desejado (nesse caso RandomForest)...

... e definimos as medidas de desempenho que desejamos calcular durante o processo de validação cruzada (acurácia, precisão, revocação e f1)

```
# PASSO 1: cria objeto da classe RandomForest
modelo = RandomForestClassifier(random_state=0)

# PASSO 2: define um dicionário com as métricas desejadas. Nesse
# caso, serão a acurácia e a precisão, recall e F1 binários
medidas = {'accuracy': make_scorer(accuracy_score),
           'prec_binary': make_scorer(precision_score,
                                     average='binary', pos_label=1),
           'rec_binary': make_scorer(recall_score,
                                     average='binary', pos_label=1),
           'f1_binary': make_scorer(f1_score, average='binary',
                                    pos_label=1)}
```

Avaliação por validação cruzada na sklearn (5/8)

PASSO 4: estimativa por validação cruzada

(3) executa a validação cruzada em k partições

- Nesse caso, $k = 10$
- No final, a função retornará um dicionário com os resultados
 - Demora um pouco para executar, afinal será feito o treino e teste de 10 modelos

```
# PASSO 3: treina um modelo RandomForest e faz a validação  
# cruzada em 10 partições (parâmetro cv = 10), computando a  
# acurácia e a precisão, recall e F1 binários  
  
resultados = cross_validate(modelo, X, Y, scoring=medidas, cv=10)
```

Avaliação por validação cruzada na sklearn (6/8)

PASSO 4: estimativa por validação cruzada

(4) agora é só visualizar os resultados.

- veja que vem tudo em um dicionário, incluindo os tempos de fit e scoring (teste)

```
print(resultados)
```



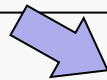
```
{'fit_time': array([5.82922816, 6.56574512, 5.69746065, 6.28974438, 5.98744035, 6.08944678, 6.23086667, 5.95523715, 6.59239674, 5.92767644]), 'score_time': array([0.05136013, 0.04460168, 0.04427433, 0.06200933, 0.04716778, 0.06383419, 0.04757118, 0.06379056, 0.04722714, 0.07167101]), 'test_accuracy': array([0.91081081, 0.94918919, 0.95081081, 0.93297297, 0.93510005, 0.91076257, 0.89075176, 0.97782585, 0.97512169, 0.97566252]), 'test_prec_binary': array([0.92649311, 0.93611111, 0.92702703, 0.92969871, 0.93123209, 0.954323 , 0.93771044, 0.95099338, 0.94591029, 0.94480946]), 'test_rec_binary': array([0.83795014, 0.93351801, 0.94882434, 0.89626556, 0.90027701, 0.81024931, 0.77146814, 0.99445983, 0.99307479, 0.99584488]), 'test_f1_binary': array([0.88 , 0.93481276, 0.93779904, 0.91267606, 0.91549296, 0.87640449, 0.84650456, 0.97224103, 0.96891892, 0.9696561 ])}
```

Avaliação por validação cruzada na sklearn (7/8)

PASSO 4: estimativa por validação cruzada

Exemplo – calculando a média e desvio padrão do f1

```
# exemplo - calculando a média e desvio padrão do f1  
print('média f1: ', resultados['test_f1_binary'].mean())  
print('desvio padrão f1: ', resultados['test_f1_binary'].std())
```



```
média f1: 0.9214505921381655  
desvio padrão f1: 0.0413269855948887
```

Avaliação por validação cruzada na sklearn (8/8)

*** IMPORTANTE

- Em muitas situações práticas, é preciso realizar um processo de **transformação de dados no treino** (**fit_transform** no vocabulário da sklearn) para **em seguida aplica-lo nos dados de teste** (**transform** no vocabulário da sklearn)
- Exemplos:
 - Mudar a escala dos dados de treino e depois aplicar no teste, para garantir que o valor máximo e mínimo usados na mudança de escala estejam no treino
 - Na PLN, gerar o tf-idf com os dados de treino e aplicar nos dados de teste, para garantir que palavras desconhecidas do teste sejam descartadas
- Para essas situações, não poderíamos utilizar o exemplo de validação cruzada mostrado nessa aula

EXERCÍCIO

- Na Matriz de Confusão abaixo, considere que **C1** é a **classe negativa** e **C2** a **classe positiva**. Calcule o valor das seguintes medidas de desempenho:
 - (a) Acurácia
 - (b) Precisão
 - (c) Revocação (*recall*)

		Rótulos Preditos	
	Classes	C1	C2
Rótulos Reais	C1	310	14
	C2	6	170

Referências

- HAN, J.; PEI, J.; TONG, H. Model Evaluation and Selection. *In*: HAN, J.; PEI, J.; TONG, H. **Data Mining: Concepts and Techniques**. 4. ed. Waltham: Morgan Kaufman, 2023. cap 6, p. 278 – 288.
- confusion_matrix. *In*: scikit-learn documentation. Disponível em < https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html#sklearn.metrics.confusion_matrix >. Acesso em: 10 set. 2026.
- cross_validate. *In*: scikit-learn documentation. Disponível em < https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_validate.html >. Acesso em: 10 set. 2026.
- Metrics and scoring: Quantifying the quality of your predictions. *In*: scikit-learn documentation. Disponível em < https://scikit-learn.org/stable/modules/model_evaluation.html >. Acesso em: 10 set. 2026.
- train_test_split. *In*: scikit-learn documentation. Disponível em < https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html >. Acesso em: 10 set. 2026.